EgoQuant: Methods and Models, part B (limits, portfolio construction, goal simulation, Risk Engine, trade check, report, advisor)

Companion of `03_METHODS_AND_MODELS.md` (part A: questionnaire, Risk Game, behaviour, preferences, market data). Status: 20 October 2026. Each section starts with a plain-language paragraph, then the detail with file names. Everything computed from history is an estimate, never a forecast.

1. What the category means: the limits table

The risk category C1 (most careful) to C5 turns into numbers. **These values are our provisional placeholders** (a banner on every report says so); an institution would replace them. The stock, gold-and-commodity and crypto caps, the product risk level and the loss limit are written in `egoquant/risk_profile/config.yaml`; Agent A keeps a copy in `egoquant/models.py` (`RiskClassPolicy`) and adds the volatility cap, the cash cap and the default stock level; a test keeps the shared columns equal (`tests/risk_profile/test_risk_profile.py::test_agent_a_class_table_matches_questionnaire`).

Category	Highest product risk level	Stocks (max)	Gold and commodities (max)	Crypto (max)	Cash (max, builder rule)	Volatility (max)	Loss in the named crash scenario (max)
C1	R1	0%	0%	0%	none	4%	5%
C2	R2	0%	0%	0%	none	7%	10%
C3	R3	40%	10%	0%	30%	10%	20%
C4	R4	70%	15%	0%	20%	13%	35%
C5	R5	90%	20%	5%	15%	17%	45%

The named crash scenario is `equity_down_30_bond_down_5`: stocks fall 30%, bonds 5%, gold and commodities 15%, crypto 60%. Four more hypothetical scenarios are always shown (stocks -20%; a +100 basis point rate shock; an inflation shock; a gold, commodity and crypto sell-off). Further limits: no single holding above 40% (the optimizer's standard limit; category C1, whose only funds are two short-Treasury funds, may hold up to 100% in one fund, and in the trade check a purchase into a holding already over 40% is a warning, not a block), instrument caps from Agent C (for example 10% per single stock), the person's cash need, and the knowledge gate for complex products. The cash cap is a portfolio-construction rule (so that a minimum-variance optimiser does not park a growth portfolio in cash); it is not applied to a person's own trades (`CLAUDE.md` decision 14).

2. Agent A: how candidate portfolios are built

In plain words: Agent A is a maths program, not a chatbot. It takes the products Agent C allows, a price history and the limits, and builds about a hundred different portfolios that all respect every hard limit.

- **Inputs:** prices of the allowed universe (about 20 years of daily history where it exists), the person's limits, interest areas, goal. Prices are validated first (sorted dates, identifiers, gaps, stale data, look-ahead) (`egoquant/agents/quant/core.py` `validate_prices`).
- **Estimates:** annualised covariance of returns (with shrinkage), asset volatilities; expected returns are an explicit assumption, **never the historical mean** (`core.py` header).

- **Construction:** long-only weights that sum to 1, solved with a constrained numerical optimiser (SciPy SLSQP, with a linear-programming feasibility step) under every hard limit. The menu has 20 points on the risk axis (stock share from zero to the category's cap) times 5 styles (lowest risk; diversified, with a penalty on concentration; equal weight; inverse volatility, "risk-balanced"; interest tilt, which gives the person's interest areas three times the weight multiple), de-duplicated to about 100 distinct candidates (`egoquant/agents/quant/agent.py` ; `AgentConfig.candidate_*`).
- **Dynamic risk budget:** when recent volatility of the market proxy is high relative to its long-run level, the volatility cap shrinks; it never grows (`dynamic_risk_budget` , `regime_low` 0.8, `regime_high` 1.25).
- **Order of authority:** Agent A proposes; Agent B (part A) can only tighten limits and screens the candidates; the Risk Engine (section 4) decides. Tests: `tests/agent_a/` (287: 104 plus 183 for the optional v2 method), `tests/system/` .
- **A second method (off by default):** risk parity, hierarchical risk parity, maximum diversification, Black-Litterman and resampled efficiency, with a 30% cap per fund. Built and tested; held back by our own acceptance rule (two of eight criteria failed). `docs/V2_DECISION_SHEET.md` , flag `EGOQUANT_PORTFOLIO_METHOD` .

3. The goal simulation ("N in 100 simulated futures")

In plain words: to say how likely a portfolio is to reach a goal, we replay the past. We build thousands of possible futures by stitching together real months of market history and ask in how many of them the goal is reached. We do not claim the past repeats; we show how much the answer changes when the history changes, and we label it hypothetical.

- **Paths:** 10,000 simulated futures, the same ones for every candidate (so differences come from the portfolios, not from luck), a fixed seed. A future chains month-long blocks (21 trading days) of real daily returns, which keeps crashes and rebounds clustered the way they happened. Each month: the portfolio grows by its block return, rebalanced to target weights; fund fees come off; the monthly contribution (or withdrawal) is added. (`egoquant/agents/quant/goal.py`)
- **Short histories:** a young fund (a 2024 Bitcoin fund) does not shorten everyone's window. Its missing early years are rebuilt by regressing it on long-history funds and resampling residual blocks, with zero alpha (Stambaugh 1997; Page 2013; Pastor and Stambaugh 1999).
- **Two bias corrections:** (1) *rate anchor:* bond and cash funds keep every real daily move but get the rate the market pays today (plus a term premium for bonds) as their average, because past averages reflect past interest rates (with the sample averages, bond-only portfolios ended below the predicted median in 100% of past five-year cases; anchored, in 42 to 54%). (2) *hindsight correction:* stocks, equity funds and crypto are picked from today's most-traded list (today's winners), so they keep every real move but get the average their sensitivity to the world stock market earned (zero alpha; Black and Litterman 1992; Brown, Goetzmann, Ibbotson and Ross 1992).
- **Regime-aware sampler (on by default):** a four-state Hidden Markov model (calm, normal, stressed, crisis), fitted offline on market history and loaded as a hash-checked JSON file (no network, no ML library at run time), decides which kind of month comes next, so crises last as long as they did. Any problem (missing or altered file, synthetic data, too little overlap) falls back to the plain block bootstrap and says why; `EGOQUANT_REGIME_MC=0` is the kill switch (`egoquant/regime/`).
- **Choosing:** the highest simulated chance of reaching the goal; among candidates within one percentage point of the best, the smaller shortfall in the worst 5% of futures, then the lower volatility (`goal.rank`).
- **Levers:** with the same futures, the model re-runs with one input changed: the monthly amount needed for a 50, 75 or 90% chance, the chance if the goal date is two or five years later, the goal amount reached in 50, 75 or 90% of futures.

- **How much the period matters:** the chosen portfolio is also run on the older half and the newer half of history only; the gap is shown. It is much larger than the simulation's own sampling error (about half a percentage point with 10,000 futures).
- **Checked against reality, in our own backtest** (rolling start dates, models refitted on earlier data only, `eval/results/regime_v2_summary.md`): the regime sampler scores better than the plain bootstrap over all dates (Brier score lower by 0.007 at five years, 0.011 at ten, intervals above zero), but **on the later confirmation dates the improvement is within the noise**, and for some portfolio types the realised share reaching the goal differs from the predicted by tens of points. Hence "simulated", never "probable". (`docs/judges/06_LIMITATIONS_AND_ROADMAP.md` 2.3)
- **Reproducibility:** the report carries input hashes and the engine and model versions; with the same inputs on the same machine and library versions the output is identical (golden tests `tests/regime/test_regime_goal.py`, recorded on the development PC). The server runs one math-library thread by default (`main.py`), which makes results repeatable on one machine; last digits, and for the optional method v2 near-equal portfolios, can still differ between machines (`docs/judges/06_LIMITATIONS_AND_ROADMAP.md` section 2.3).

4. The Risk Engine: the independent final check

In plain words: a second, separate program re-checks the finished portfolio from scratch. If it finds a problem, the portfolio is not shown. It does not trust the optimiser.

It does not call Agent A, Agent B or Agent A's maths; it recomputes everything from the proposal's weights, the products' metadata and the limits (`egoquant/risk_engine.py`, version 0.2.0). Checks: 1. **Weights:** long-only and summing to 1. 2. **Product risk level:** every holding's level is at most the person's maximum; an unknown ticker or a missing level is a violation (**fails closed**); no holding above its instrument cap. 3. **Stocks:** total stock weight within every hard limit. 4. **Class caps:** commodity, crypto and cash within their caps. 5. **Stress loss:** the loss in each named hypothetical scenario within every hard limit.

Soft constraints are not enforced here (Agent A may relax them by design, and the report says when it did). When the engine rejects, the person sees plain words ("no portfolio fits all of your limits at once; a high cash or liquidity need is the usual cause"), never a code. Tests: `tests/system/test_risk_engine.py`, `tests/test_plain_wording.py`.

5. The trade check

In plain words: "I want to buy 3,000 of this fund. Does that fit my limits?" The answer is computed, not guessed, and it never says "do it" or "don't".

- **Method:** the person's holdings (from the saved trading history, never from the request) plus the planned trade give an after-trade portfolio; it is judged against the same limits the Risk Engine checks (suitability, category caps, product risk levels, Agent B's hard limits) and re-reviewed by the Risk Engine. A buy is paid from account cash; a shortfall counts as new money and is said so. A sale's proceeds stay as cash.
- **Verdicts:** *advisable* (no limit broken or worsened), *allowed with warnings* (nothing newly broken or worsened, but something to know: a limit still exceeded that this trade reduces, a soft limit, little room left, an expired profile), *not advisable* (a hard limit broken, or an existing breach made worse). On the page these labels are the check's own; **the chat states them only as "within your limits", "within your limits, with warnings" or "outside your limits"** (`verdict_words`).
- **Every reason carries numbers** (limit, before, after); the explanation text only phrases them; it quotes nothing that is not in the check. Property-tested on 5,000 random trades

(`tests/system/test_trade_check_sweep.py`); the chat tool runs the same code (`egoquant/agents/trade_chat.py`).

6. The report

A client report (JSON and a self-contained HTML page) from fixed templates: the headline, the category and **how it was set** (`category_basis`), the allocation, each limit with its source and where it stands, the reasons, the Risk Engine's checks, five stress scenarios, fees and trading costs, the goal analysis, the data sources and dates, notes, and the mandatory disclosures (hypothetical, in-sample, "DRAFT LIMITS" banner, not a forecast). Wording is plain ("your category", "attitude to risk"); sentences from the Risk Engine and the trade check are rewritten into plain language by tested rules (`egoquant/report.py`, `egoquant/risk_engine.py` `plain_violations`). The HTML equals the JSON by construction (a test compares them).

7. The advisor (Ego)

In plain words: a chat assistant that explains the report in the person's own words, can try "what if" scenarios and check one possible trade, and is fenced in so that it cannot decide anything or invent a number.

- **Model and API:** OpenAI Responses API (default model set by `OPENAI_MODEL`), `store=false`; the model sees only the engine's facts (HMAC-sealed by the server), never balances, holdings lists or raw trades.
- **Two tools:** `run_what_if` re-runs the deterministic engine on a validated scenario (extra monthly amount, one-off amount, horizon, goal, excluded products, mode); `check_trade` runs the trade check above (signed-in investors with a saved history, a trade-check consent and a portfolio built while signed in). Whitelisted arguments (no field for limits, categories or identities), at most two calls per message, results behind number grounding.
- **Number grounding** (`egoquant/advisor/grounding.py`): every percentage, "N in 100", money amount and multiplier must come from the facts, this turn's tool results or the user's words; otherwise regenerate once, then a safe fallback.
- **Redaction gateway** (`egoquant/advisor/redaction.py`): identifiers and the person's name become placeholders on the way out (the first name is the token `[NAME_1]`) and are restored in the reply.
- **Compliance guard** (`egoquant/advisor/guard.py`, `config.yaml`): withholds sentences that read like trade instructions, price targets, guarantees or forecasts (36 patterns, English and Chinese), sentence by sentence, also while streaming.
- **Prompt versions** are logged with every call (`advisor-v11` today: trade questions, personalization, report-tab awareness, verdict words, one size field for the trade tool). The red-team harness (`eval/chat_redteam.py`) has cases for what-if, regimes, compliance and 18 for trade questions; earlier versions passed the live runs kept in `eval/results/` (advisor chat, prompts v2 to v5: 19/19, 19/19, 21/21 and 24/24; trade explanation: 22/24 then 24/24, 24/24 and 51/51 twice). **On 20 Oct the live red-team of advisor-v10 found a real defect in the trade tool** (the model filled both size fields, so no trade check completed; a safe failure), **fixed the same hour in advisor-v11, which passed 41 of 42 cases** (24 of 24 non-trade, 17 of 18 trade; the miss is a false positive of the keyword check); **trade-explain-v4 passed 51 of 51** (`eval/results/*_20261020_*.jsonl`).

8. Other components, briefly

- **Agent D (news briefing, Aslan):** news per holding and interest area, an AI tone label per headline and fund (85% agreement with 120 hand labels; FinBERT 53%), source tiers, "what changed"; never changes a number (`egoquant/briefing/README.md` , `docs/AGENT_D_LIMITATIONS.md`).
- **On-chain seal (Aslan):** at confirm the final report is rendered once into a sealed document ("Sealed for: <username>"); a salted SHA-256 fingerprint of the advice record (which includes the document's own SHA-256 and the username) goes on the Sepolia test chain; the public **Verify a report** page checks a downloaded sealed report in the browser, with no account (`docs/ANCHORING.md` , `docs/ONCHAIN_DECISIONS.md`).
- **Accounts:** Supabase Auth and Postgres with Row-Level Security (`docs/judges/02_SECURITY_COMPLIANCE_REPORT.md` sections 4 and 6).